



RecyclerView

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet
2025/2026

Cos'è RecyclerView

RecyclerView è il widget Android per visualizzare elenchi grandi e scorrevoli in modo efficiente. Ricicla le view che escono dallo schermo per riusarle con nuovi dati.

- Ricicla le view: solo quelle visibili + un piccolo buffer vengono create in memoria.
- Separa i dati (Adapter) dalla presentazione (ViewHolder) dal layout (LayoutManager).
- Supporta liste verticali, orizzontali, griglie e layout custom.
- Animazioni di inserimento, rimozione e spostamento integrate.
- Sostituisce ListView e GridView: più flessibile, più performante.
- Parte di AndroidX: implementazione stabile e retrocompatibile.



RecyclerView vs ListView

ListView crea una view per ogni elemento → spreco di memoria. RecyclerView ricicla le view → $O(\text{visibili})$ view in memoria, non $O(n)$.

Architettura: i tre componenti

RecyclerView delega la responsabilità a tre oggetti separati: Adapter, ViewHolder e LayoutManager.



Adapter

Fornisce i dati e crea i ViewHolder. È il ponte tra la lista di dati e le view. Metodi chiave: `onCreateViewHolder()`, `onBindViewHolder()`, `getItemCount()`.



ViewHolder

Contiene i riferimenti alle view di una singola riga (binding). Viene riusato: il riciclo avviene qui. Evita chiamate ripetute a `findViewById`.



LayoutManager

Decide come disporre gli elementi: lista verticale (`LinearLayoutManager`), griglia (`GridLayoutManager`), griglia sfalsata (`StaggeredGridLayoutManager`).

Il modello dati

Ogni elemento della lista è rappresentato da una data class. L'Adapter riceve una lista di questi oggetti.

```
// Modello dati semplice
data class Contact(
    val id: Long,
    val name: String,
    val email: String,
    val avatarUrl: String? = null
)
```

Kotlin

```
// Modello più complesso
data class Product(
    val id: Int,
    val name: String,
    val price: Double,
    val imageUrl: String,
    val category: String,
    val inStock: Boolean = true
)
```

Kotlin



Perché data class?

data class genera automaticamente equals(), hashCode() e copy(). DiffUtil usa equals() per capire se un elemento è cambiato.



Ogni data class dovrebbe avere un campo id univoco: serve come chiave stabile per DiffUtil e per le animazioni.

Layout XML della riga

Ogni elemento della lista ha il proprio layout XML. Va tenuto il più piatto possibile (evita nesting).

```
<!-- res/layout/item_contact.xml -->
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="2dp">

    <LinearLayout
        android:orientation="horizontal"
        android:padding="12dp"
        android:gravity="center_vertical"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <ImageView android:id="@+id/avatar"
            android:layout_width="48dp"
            android:layout_height="48dp"/>

        <LinearLayout android:orientation="vertical"
            android:layout_marginStart="12dp"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1">

            <TextView android:id="@+id/name" .../>
            <TextView android:id="@+id/email" .../>
        </LinearLayout>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
```

XML

Anteprima**Contatto 1**

email1@example.com

**Contatto 2**

email2@example.com

**Contatto 3**

email3@example.com

**Contatto 4**

email4@example.com

ViewHolder con ViewBinding

Il ViewHolder mantiene i riferimenti alle view della riga. Con ViewBinding non serve findViewById.

```
class ContactViewHolder(  
    private val binding: ItemContactBinding  
) : RecyclerView.ViewHolder(binding.root) {  
  
    fun bind(contact: Contact, onClick: (Contact) -> Unit) {  
        binding.name.text = contact.name  
        binding.email.text = contact.email  
  
        // Carica l'avatar con Coil/Glide  
        contact.avatarUrl?.let { url ->  
            binding.avatar.load(url) {  
                crossfade(true)  
                placeholder(R.drawable.ic_person)  
            }  
        }  
  
        // Click listener sulla riga intera  
        binding.root.setOnClickListener {  
            onClick(contact)  
        }  
    }  
}
```

Kotlin

L'Adapter: il ponte tra dati e view

L'Adapter implementa tre metodi fondamentali: crea il ViewHolder, lo popola con i dati, e dichiara quanti elementi ci sono.

```
class ContactAdapter(  
    private var contacts: List<Contact>,  
    private val onClick: (Contact) -> Unit  
) : RecyclerView.Adapter<ContactViewHolder>() {  
  
    // 1. Crea un nuovo ViewHolder (inflating del layout)  
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ContactViewHolder {  
        val binding = ItemContactBinding.inflate(  
            LayoutInflater.from(parent.context), parent, false  
        )  
        return ContactViewHolder(binding)  
    }  
  
    // 2. Popola il ViewHolder con i dati della posizione  
    override fun onBindViewHolder(holder: ContactViewHolder, position: Int) {  
        holder.bind(contacts[position], onClick)  
    }  
  
    // 3. Ritorna il numero totale di elementi  
    override fun getItemCount(): Int = contacts.size  
}
```

Kotlin

Setup nella Activity / Fragment

Nella Activity o Fragment si collega RecyclerView, LayoutManager e Adapter.

activity_main.xml

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layoutManager="LinearLayoutManager"/>
```

XML



LayoutManager da XML o da codice

Puoi impostare app:layoutManager in XML o
binding.recyclerView.layoutManager =
LinearLayoutManager(this).

MainActivity.kt

```
class MainActivity : AppCompatActivity() {
    private lateinit var binding:
        ActivityMainBinding

    override fun onCreate(b: Bundle?) {
        super.onCreate(b)
        binding = ActivityMainBinding
            .inflate(layoutInflater)
        setContentView(binding.root)

        val contacts = loadContacts()

        val adapter = ContactAdapter(contacts) {
            // click su un contatto
            Toast.makeText(this,
                it.name, Toast.LENGTH_SHORT).show()
        }

        binding.recyclerView.adapter = adapter
    }
}
```

Kotlin

LayoutManager: tipi di layout

Il LayoutManager decide come posizionare gli elementi nella RecyclerView.



LinearLayoutManager

Lista verticale o orizzontale. Il più comune.

```
LinearLayoutManager(context)
// Orizzontale:
LinearLayoutManager(
    context,
    LinearLayoutManager.HORIZONTAL,
    false
)
```

Kotlin



GridLayoutManager

Griglia con N colonne fisse.

```
GridLayoutManager(context, 2)
// 2 colonne
// span size per header:
grid.spanSizeLookup = object :
    GridLayoutManager.SpanSizeLookup() {
        override fun getSpanSize(pos: Int) =
            if (pos == 0) 2 else 1
    }
}
```

Kotlin



StaggeredGridLayoutManager

Griglia con elementi di altezza variabile (Pinterest-style).

```
StaggeredGridLayoutManager(
    2,
    StaggeredGridLayoutManager.VERTICAL
)
```

Kotlin

Gestire i click

Il pattern consigliato è passare un lambda al costruttore dell'Adapter, poi delegare nel ViewHolder.

```
// Nell'Adapter
class ContactAdapter(
    private var contacts: List<Contact>,
    private val onClick: (Contact) -> Unit,
    private val onLongClick: (Contact) -> Boolean
) : RecyclerView.Adapter<ContactViewHolder>() {
    // ...
    override fun onBindViewHolder(
        holder: ContactViewHolder, position: Int
    ) {
        val contact = contacts[position]
        holder.bind(contact, onClick, onLongClick)
    }
}

// Nel ViewHolder
fun bind(
    contact: Contact,
    onClick: (Contact) -> Unit,
    onLongClick: (Contact) -> Boolean
) {
    // ...
    binding.root.setOnClickListener { onClick(contact) }
    binding.root.setOnLongClickListener {
        onLongClick(contact)
    }
}
```

Kotlin

Pattern dei click

- Lambda nel costruttore dell'Adapter
- Delegate nel bind() del ViewHolder
- MAI usare activity reference nell'Adapter
- Il click handler vive nella Activity/Fragment
- onLongClick ritorna Boolean (consumato?)
- Per singole azioni: listener sul bottone specifico

Swipe-to-delete con ItemTouchHelper

ItemTouchHelper.SimpleCallback permette di aggiungere gesti di swipe e drag-and-drop alla RecyclerView.

```
val swipeHandler = object : ItemTouchHelper.SimpleCallback(
    0, // drag directions (0 = nessun drag)
    ItemTouchHelper.LEFT or ItemTouchHelper.RIGHT // swipe
) {
    override fun onMove(rv: RecyclerView, vh: RecyclerView.ViewHolder,
        target: RecyclerView.ViewHolder) = false // no drag

    override fun onSwiped(viewHolder: RecyclerView.ViewHolder, direction: Int) {
        val position = viewHolder.adapterPosition
        val contact = contacts[position]
        contacts = contacts.toMutableList().also { it.removeAt(position) }
        adapter.notifyItemRemoved(position)

        // Snackbar con Undo
        Snackbar.make(binding.root, "${contact.name} rimosso", Snackbar.LENGTH_LONG)
            .setAction("Annulla") {
                contacts = contacts.toMutableList().also { it.add(position, contact) }
                adapter.notifyItemInserted(position)
            }.show()
    }
}

ItemTouchHelper(swipeHandler).attachToRecyclerView(binding.recyclerView)
```

Kotlin

DiffUtil: aggiornamenti intelligenti

DiffUtil calcola la differenza tra due liste e produce le operazioni minime di aggiornamento con animazioni automatiche.

```
class ContactDiffCallback : DiffUtil.ItemCallback<Contact>() {  
  
    // Stesso elemento? Confronta l'ID  
    override fun areItemsTheSame(old: Contact, new: Contact): Boolean =  
        old.id == new.id  
  
    // Stesso contenuto? Confronta tutti i campi (data class equals)  
    override fun areContentsTheSame(old: Contact, new: Contact): Boolean =  
        old == new  
  
}
```

Kotlin

Perché non notifyDataSetChanged()?

	notifyDataSetChanged()	DiffUtil
Complessità	Ricostruisce tutto	Solo le differenze
Animazioni	Nessuna	Automatiche (fade, slide)
Performance	O(n) inutile	O(n) mirato

ListAdapter: DiffUtil integrato

ListAdapter è una sottoclasse di RecyclerView.Adapter che integra DiffUtil in modo automatico. È il modo consigliato.

```
class ContactListAdapter(  
    private val onClick: (Contact) -> Unit  
) : ListAdapter<Contact, ContactViewHolder>(ContactDiffCallback()) {  
  
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ContactViewHolder {  
        val binding = ItemContactBinding.inflate(  
            LayoutInflater.from(parent.context), parent, false  
        )  
        return ContactViewHolder(binding)  
    }  
  
    override fun onBindViewHolder(holder: ContactViewHolder, position: Int) {  
        holder.bind(getItem(position), onClick) // getItem() da ListAdapter  
    }  
}  
  
// Nella Activity: aggiornare la lista e basta!  
val adapter = ContactListAdapter { contact -> /* click */ }  
binding.recyclerView.adapter = adapter  
adapter.submitList(newContacts) // DiffUtil calcola le differenze automaticamente
```

Kotlin

ItemDecoration: divisori e spaziatura

Le decorazioni aggiungono spaziatura, divisori o effetti visivi agli elementi senza modificare il layout.

```
// Divisore Material
val divider = MaterialDividerItemDecoration(
    context, LinearLayoutManager.VERTICAL
).apply {
    dividerInsetStart = 80 // inset sinistro (px)
    isLastItemDecorated = false
}
binding.recyclerView.addItemDecoration(divider)
```

Kotlin

```
// Spaziatura custom
class SpacingDecoration(private val space: Int)
    : RecyclerView.ItemDecoration() {
    override fun getItemOffsets(
        out: Rect, v: View, rv: RecyclerView,
        state: RecyclerView.State
    ) { out.bottom = space }
}
rv.addItemDecoration(SpacingDecoration(16))
```

Kotlin

Tipi di decoration

- DividerItemDecoration: linea divisoria standard
- MaterialDividerItemDecoration: con inset e colore Material
- Custom: spaziatura, bordi, ombre, header sticky
- Più decoration combinabili con addItemDecoration()

Multiple view types

Una RecyclerView può mostrare righe di tipo diverso (es. header + contenuto + pubblicità).

```
sealed class ListItem {
    data class Header(val title: String) : ListItem()
    data class ContactItem(val contact: Contact) : ListItem()
}

class MultiAdapter : RecyclerView.Adapter<RecyclerView.ViewHolder>() {
    private var items: List<ListItem> = emptyList()

    companion object {
        const val TYPE_HEADER = 0
        const val TYPE_CONTACT = 1
    }

    override fun getItemViewType(position: Int) = when (items[position]) {
        is ListItem.Header -> TYPE_HEADER
        is ListItem.ContactItem -> TYPE_CONTACT
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int) = when (viewType) {
        TYPE_HEADER -> HeaderViewHolder(/* inflate header layout */)
        else -> ContactViewHolder(/* inflate contact layout */)
    }

    override fun onBindViewHolder(holder: RecyclerView.ViewHolder, position: Int) {
        when (val item = items[position]) {
            is ListItem.Header -> (holder as HeaderViewHolder).bind(item.title)
            is ListItem.ContactItem -> (holder as ContactViewHolder).bind(item.contact)
        }
    }

    override fun getItemCount() = items.size
}
```

Kotlin

RecyclerView vs LazyColumn (Compose)

RecyclerView è il sistema View. LazyColumn è l'equivalente in Compose: stesso principio, API diversa.

Aspetto	RecyclerView	LazyColumn
Paradigma	Imperativo (XML + Kotlin)	Dichiarativo (Compose)
Setup	Adapter + ViewHolder + Layout XML	items { } con composable
Aggiornamenti	DiffUtil / ListAdapter	Ricomposizione automatica
Click	Lambda nell'Adapter	Modifier.clickable { }
ViewType multipli	getItemViewType + sealed class	if/when nel corpo di items
Complessità	~100 righe per lista base	~15 righe per lista base
Performance	Eccellente (maturo)	Eccellente (da Compose 1.2+)
Quando usarlo	Progetti View esistenti	Nuovi progetti Compose



In nuovi progetti Compose, LazyColumn sostituisce RecyclerView. In progetti View, RecyclerView resta la scelta giusta.

Best practice

DA FARE

- Usa ListAdapter con DiffUtil
- ViewBinding nel ViewHolder
- setHasFixedSize(true) se la lista non cambia altezza
- Click via lambda, non reference ad Activity
- Usa data class con id univoco
- RecyclerViewPool per RecyclerView annidate
- Carica immagini con Coil/Glide (asincrono)

DA EVITARE

- notifyDataSetChanged() per ogni modifica
- findViewById nel ViewHolder
- Layout XML annidati a profondità 4+
- Logica pesante in onBindViewHolder
- Creare oggetti dentro onBind (GC pressure)
- Adapter che tiene riferimento all'Activity
- Dimenticare setHasStableIds con DiffUtil

Errori comuni e soluzioni

Posizione sbagliata nel click

Usare `holder.adapterPosition` (non il parametro `position` di `onBind`, può essere stale).

RecyclerView non scorre

Verificare che il layout della riga abbia `wrap_content` in altezza, non `match_parent`.

Lista vuota senza feedback

Mostrare un `EmptyState` view quando la lista è vuota. Usare `adapter.itemCount == 0`.

`notifyDataSetChanged` troppo usato

Usare `ListAdapter.submitList()` o i metodi specifici (`notifyItemInserted`, ecc.).

Immagini che "saltano" nello scroll

Il `ViewHolder` è riciclato: cancellare l'immagine prima di caricarne una nuova.

Click che si duplica

Non aggiungere listener in `onBind` senza rimuovere il precedente. Meglio: impostarlo una volta in `onCreateViewHolder`.

Cheat-sheet RecyclerView

Kotlin

```
// 1. Data class
data class Contact(val id: Long, val name: String, val email: String)

// 2. DiffUtil
class ContactDiff : DiffUtil.ItemCallback<Contact>() {
    override fun areItemsTheSame(o: Contact, n: Contact) = o.id == n.id
    override fun areContentsTheSame(o: Contact, n: Contact) = o == n
}

// 3. ListAdapter (consigliato)
class ContactAdapter(val onClick: (Contact) -> Unit)
    : ListAdapter<Contact, ContactVH>(ContactDiff()) {
    override fun onCreateViewHolder(p: ViewGroup, vt: Int) = ContactVH(/*binding*/)
    override fun onBindViewHolder(h: ContactVH, pos: Int) = h.bind(getItem(pos), onClick)
}

// 4. Setup
binding.recyclerView.adapter = adapter
binding.recyclerView.layoutManager = LinearLayoutManager(this)
adapter.submitList(contacts) // DiffUtil automatico

// 5. Aggiornare
adapter.submitList(newList) // animazioni automatiche

// 6. Decoration
binding.recyclerView.addItemDecoration(MaterialDividerItemDecoration(ctx, VERTICAL))
```